

Métodos numéricos contra la forma cerrada

Lote económico de pedido · Unidad 2

Bryan Rodríguez Abarca · 6 de septiembre de 2026

1. El problema con números

La instancia que quedó apartada en la actividad anterior cuesta $K = 450$ por pedido colocado, mueve $D = 12\,000$ unidades al año y paga $h = 6$ por guardar una unidad durante ese año; el almacén no acepta lotes menores de 50 ni mayores de 2000. La variable de decisión es el tamaño del lote Q , en unidades y continua; el objetivo, el costo anual que suma lo que cuesta pedir D/Q veces y lo que cuesta guardar un inventario medio de $Q/2$ (Harris, 1913).

$$\min_Q C(Q) = \frac{KD}{Q} + \frac{hQ}{2}, \quad 50 \leq Q \leq 2000.$$

El planteamiento es el de la investigación previa, que se detuvo antes de la fórmula. Aquí hace falta escribirla, porque es la vara contra la que se mide todo lo demás:

$$C'(Q) = -\frac{KD}{Q^2} + \frac{h}{2} = 0 \implies Q^* = \sqrt{\frac{2KD}{h}}, \quad C''(Q) = \frac{2KD}{Q^3}.$$

El cociente $2KD/Q^3$ se mantiene por encima de cero mientras Q lo esté, así que C es estrictamente convexa sobre ese semieje y su único punto estacionario alcanza el mínimo global (Nocedal y Wright, 2006). Las cotas del almacén se leen con Karush-Kuhn-Tucker, y con objetivo convexo y una sola incógnita eso equivale a recortar Q^* contra el intervalo (Antonioni y Lu, 2007). Con estos datos $Q^* = 1341.640786499874$ cae dentro, las dos cotas quedan inactivas y el costo del óptimo vale 8049.844718999243.

2. Cómo se detiene cada rutina

La sección áurea y el método de Brent salen de `minimize_scalar` (The SciPy community, 2024) y trabajan igual en lo esencial: encierran el mínimo entre tres puntos y estrechan el intervalo leyendo únicamente valores de C . Brent alterna ese estrechamiento con una interpolación parabólica sobre los tres últimos puntos, que acelera cuando el objetivo se parece a una parábola —cerca del mínimo se le parece mucho— y se replica sobre la sección áurea cuando no (Brent, 1973). Newton va escrito a mano porque consume las dos derivadas y `minimize_scalar` no admite ninguna; aquí ambas son exactas y cuestan lo mismo que evaluar el objetivo.

```
def newton(Q0, tol=1e-12, maxit=100):
    Q = float(Q0)
    for i in range(maxit):
        Qn = Q - dC(Q) / d2C(Q)
        if abs(Qn - Q) <= tol * abs(Qn):
            return Qn, i + 1, 2 * (i + 1)
        Q = Qn
    return Q, maxit, 2 * maxit

res_aurea = minimize_scalar(C, bracket=(QMIN, QMAX), method="golden")
res_brent = minimize_scalar(C, bracket=(QMIN, QMAX), method="brent")
Q_newton, it_newton, ev_newton = newton(QMIN)
```

Newton arranca en el lote mínimo de 50 unidades y el `bracket` de los otros dos empieza en ese mismo punto. Las tres corridas de la tabla usan el valor por omisión de cada rutina: `minimize_scalar`

para cuando el intervalo baja de $\text{xtol} = \sqrt{\varepsilon} = 1.4901 \times 10^{-8}$, y el `newton` de arriba cuando el último paso baja de 10^{-12} , cuatro órdenes más apretado. Repetido con el número de SciPy, Newton cae en el mismo double tras las mismas 14 iteraciones, así que esa distancia no mueve ninguna celda. Donde los tres sí reciben el mismo número es en el barrido, y ahí ese número manda sobre cantidades distintas —el ancho del intervalo en dos casos, la longitud del último paso en el otro—; el criterio de Brent tampoco es del todo relativo, porque suma un piso absoluto de 10^{-11} a $\text{xtol} \cdot |Q|$ que pesa en el extremo apretado.

Pasar las cotas del almacén como `bracket=(50, 2000)` me costó la variante de capacidad recortada. Con `bracket=(50, 1200)` Brent devuelve 1341.6407981341317 y `success` en `True`: un lote que el almacén no puede guardar. Ese par no acota nada; es el arranque de un rastreo que busca encerrar al mínimo y avanza hasta donde haga falta, y el envoltorio que puse alrededor de C para seguirle la pista registró llamadas en $Q = 5155.1663$ con los dos métodos. Medir la capacidad recortada obligó a pasarla a `method="bounded"`, cuya tolerancia es absoluta, y con eso quedó fuera del barrido de precisión: se trata aparte y sin curva propia.

3. El costo se repite, el trabajo no

	Q obtenido	evaluaciones	ms (mediana)	error en Q	error en C
Sección áurea	1341.6407889094910	44	0.0765	2.4096×10^{-6}	1.0690×10^{-13}
Brent	1341.6407862633980	16	0.0665	2.3648×10^{-7}	1.0690×10^{-13}
Newton	1341.6407864998737	28 [†]	0.0056	1.3377×10^{-13}	1.0690×10^{-13}
Forma cerrada	1341.6407864998739	—	0.0001	9.3607×10^{-14}	1.0690×10^{-13}

[†] evaluaciones de derivada, dos por iteración; Newton no evalúa C ni una vez.

La columna del costo es la misma en las cuatro filas y no por redondeo del informe: los cuatro valores de Q producen el mismo double, 8049.844718999243, aunque el primero y el tercero se separen 2.4 millonésimas de unidad. La comparación entre métodos, entonces, ocurre entera en la variable; el objetivo no los distingue.

Las cuentas de trabajo sí los separan y no cambian de una ejecución a otra: 39 iteraciones y 44 llamadas a C en la sección áurea, 13 y 16 en Brent, 14 iteraciones de Newton que gastan 28 evaluaciones de derivada. Los dos primeros números son comparables entre sí y el tercero está en otra moneda, porque una derivada no cuesta lo mismo que un valor de la función en un objetivo cualquiera —en este sí, pero es una particularidad del EOQ, no una regla.

El tiempo por llamada es la mediana de siete tandas de doscientas llamadas cada una en un equipo con macOS y procesador ARM. Repetí ese protocolo diez veces: Brent quedó por debajo de la sección áurea en las diez, con la razón de medianas entre 1.118 y 1.162. Esa razón no se parece al 2.75 de las evaluaciones porque la columna de milisegundos mide sobre todo al intérprete: las 44 llamadas a C de la sección áurea suman 6.2 μs de los 75 que tarda la corrida entera, y el resto es armazón de `minimize_scalar`, idéntico para los dos métodos. Escribiendo el mismo costo como suma sobre 300 tramos de demanda —una evaluación que sí pesa— la razón sube a 2.61 y se acerca al cociente de evaluaciones. El factor 12 de Newton sobre Brent mide en buena parte la ausencia de ese armazón, y evaluar la raíz cuadrada, en 0.0001 ms, deja a los tres fuera de escala.

4. Dónde se separan del valor exacto

No coincide. El valor analítico contra el que se compara tampoco es exacto, así que la medición empieza por ahí. $Q^* = \sqrt{2KD/h}$ es irracional —vale $600\sqrt{5}$ — y ningún float64 lo representa. Evaluado con mpmath a 50 dígitos da 1341.640786499873817845504..., y la fórmula en aritmética de doble precisión devuelve un valor que queda a 9.36071×10^{-14} de ahí, o 6.97706×10^{-17} en relativo. El salto entre doubles vecinos en esa magnitud es 2.2737×10^{-13} , y esa distancia no llega a la mitad del salto, así que la fórmula entrega el double más cercano al valor verdadero y no le queda margen para mejorar.

Newton devuelve 1341.6407864998737, el double contiguo por abajo —la diferencia con la fórmula es exactamente un salto— y Q^* cae entre los dos, sin ser ninguno de ellos. La sección áurea y Brent se

quedan mucho antes, a 2.4096×10^{-6} y 2.3648×10^{-7} , seis o siete órdenes por encima de ese piso, y aun así los cuatro devuelven el mismo costo.

El motivo se mide sin salir del objetivo. Su curvatura en el óptimo vale $C''(Q^*) = h/Q^* = 0.004472136$, tan pequeña que el término cuadrático se hunde bajo la resolución del punto flotante: desplazar Q en 1×10^{-5} deja C bit a bit igual, 2×10^{-5} lo mueve exactamente un salto entre doubles vecinos (9.0949×10^{-13} , un ulp) y hace falta llegar a 10^{-4} para moverlo 25. Muestreando 10 001 puntos en $Q^* \pm 10^{-5}$, C toma tres valores distintos y la amplitud entre el mayor y el menor equivale a dos ulps; sobre una rejilla más fina hay desplazamientos de hasta 1.57×10^{-5} que no cambian nada.

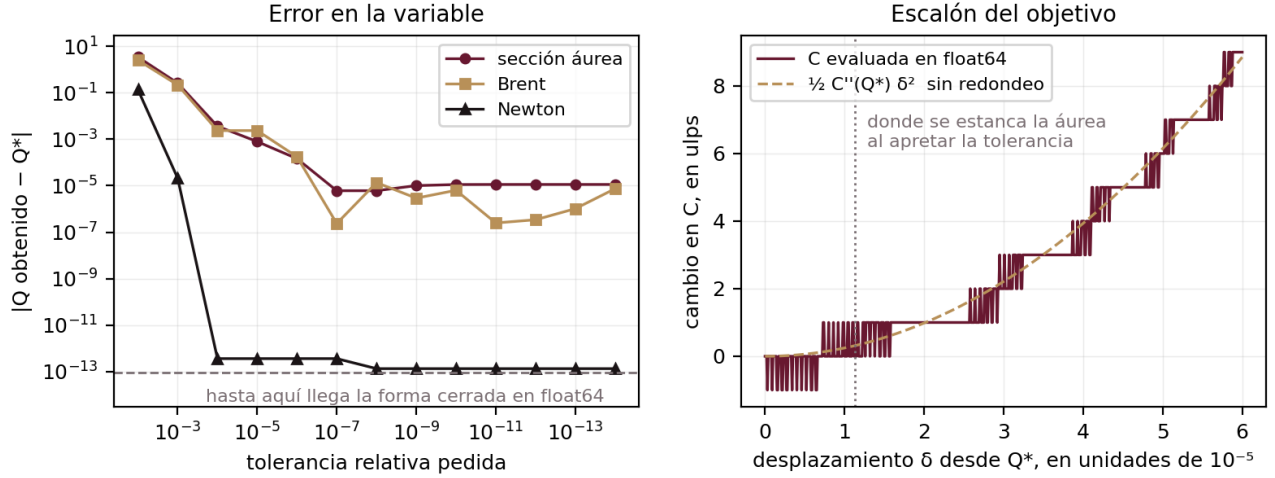


Figura 1: Izquierda: error en la variable contra la tolerancia pedida, con la línea de puntos en el piso de la forma cerrada. Derecha: cambio del costo al alejarse del óptimo, medido en saltos entre doubles vecinos, contra el término cuadrático sin redondeo; los peldaños por debajo de cero son los puntos donde la C evaluada cae bajo su valor en Q^* .

El barrido confirma dónde deja de mandar la tolerancia en los métodos de intervalo. Hasta 1×10^{-7} apretarla baja el error; a partir de ahí no. La sección áurea pasa de 40 a 74 evaluaciones entre 10^{-7} y 10^{-14} y su error se queda entre 6.11×10^{-6} y 1.135×10^{-5} , sin bajar; de 10^{-8} en adelante el de Brent salta entre 2.50×10^{-7} y 1.37×10^{-5} sin seguir a la tolerancia —su mejor valor de esa zona no es el de la tolerancia más apretada—, de modo que ordenar los dos métodos por precisión ahí sería leer ruido. Newton alcanza 3.61×10^{-13} ya con tolerancia 10^{-4} y 1.34×10^{-13} desde 10^{-8} , y no se mueve de ahí.

La coincidencia se parte según dónde se mire. En el objetivo es exacta hasta el último bit, y lo que la produce es el redondeo, que borra la diferencia entre los métodos antes de que ellos alcancen a marcarla. En la variable no hay coincidencia. A Newton lo detiene la representación: el número que busca no existe en float64. A la sección áurea y a Brent los detiene algo peor que un objetivo plano. Sobre 12 001 puntos en $Q^* \pm 6 \times 10^{-5}$ la C evaluada toma once valores, y en 342 de ellos —todos con $|\delta| \leq 6.84 \times 10^{-6}$ — queda un ulp por debajo del valor en la forma cerrada, aunque ese double esté más lejos del C^* exacto (8.03×10^{-13} contra 1.07×10^{-13}). Leen una escalera cuyos peldaños bajan y vuelven a subir, y cada uno de esos 342 puntos es un mínimo legítimo para una rutina que solo compara valores. De ahí que el error de Brent salte sin seguir a la tolerancia: dónde para depende de a qué peldaño llegue primero.

5. Cuando la cota se activa

Bajar la capacidad a 1200 saca a Q^* del intervalo y manda el óptimo al borde: el recorte da 1200 exacto y un costo de 8100.0, un 0.62306 % por encima del óptimo libre, que reproduce el 0.62 % que la actividad anterior había obtenido por la vía de la razón de costos. `bounded` llega a 1199.9999786882763 con 37 evaluaciones, le faltan 2.13×10^{-5} para tocar la cota y su costo sale 8100.000015983795. Ese exceso de 1.5983795×10^{-5} es el error en Q multiplicado por la pendiente del objetivo en el borde,

$C'(1200) = -0.75$: el producto vale 1.5983793×10^{-5} . Donde el mínimo es interior la derivada se anula y un error en Q se paga al cuadrado; sobre una cota activa la derivada vale -0.75 y el mismo error se paga en línea recta. Ese error de 2.13×10^{-5} en el lote, que alrededor del óptimo interior habría movido el costo un ulp, aquí sale en el sexto decimal.

6. Cuál usaría y con qué condición

Newton llega más cerca y más rápido que los otros dos, y cobra por ello. La derivada segunda aquí sale de una línea, y en un objetivo tabulado o con ruido de medición no sale de ninguna. Su criterio, además, es $C'(Q) = 0$ y nada más: el siguiente punto vale $Q(3/2 - Q^2/(2Q^{*2}))$ y se vuelve negativo en cuanto Q pasa de $\sqrt{3}Q^* = 2323.79$. Arrancando en 2324 converge a -1341.640786499874 , donde la derivada segunda es negativa. Con la capacidad de este almacén el arranque natural cae dentro de la cuenca; con una de 2500 no.

La sección áurea gasta 44 evaluaciones para quedarse donde Brent se queda con 16, y no compra precisión con la diferencia. Lo que ofrece a cambio es no depender de que el objetivo se parezca a una parábola cerca del óptimo; este se parece tanto que la interpolación de Brent acierta casi siempre, así que la garantía se paga sin usarla. Para el problema de esta actividad la forma cerrada gana en tiempo, en trabajo y en exactitud, y la comparación numérica sigue teniendo sentido porque el mismo modelo pierde la forma cerrada en cuanto entran descuentos por volumen o la demanda cambia de un periodo al siguiente.

La precisión alcanzada llega a 1.34×10^{-13} en el lote con Newton —a un solo double de la forma cerrada— y se queda en el orden de 10^{-6} con los métodos de intervalo. En el costo los cuatro caminos aciertan 1.0690×10^{-13} , cifra que no mide a ningún método sino el redondeo de evaluar C una vez. Lo que este banco de pruebas no alcanza a ordenar es la precisión de la sección áurea frente a la de Brent por debajo de 10^{-5} : ahí las dos devuelven puntos que el objetivo no distingue.

Referencias

- Antonioni, A., & Lu, W.-S. (2007). *Practical optimization: Algorithms and engineering applications*. Springer.
- Brent, R. P. (1973). *Algorithms for minimization without derivatives*. Prentice-Hall.
- Harris, F. W. (1913). How many parts to make at once. *Factory, The Magazine of Management*, 10(2), 135–136, 152.
- Nocedal, J., & Wright, S. J. (2006). *Numerical optimization* (2nd ed.). Springer.
- The SciPy community. (2024). *scipy.optimize.minimize_scalar* [Documentación de SciPy 1.13.1]. https://docs.scipy.org/doc/scipy-1.13.1/reference/generated/scipy.optimize.minimize_scalar.html